

THEORETICAL REPORT

MODULE 4 – ROBOT FRAMEWORK

Name: Saheli Deb

Enrollment No.: 12023052019071

Course: Python Automation

Report Type: Section-wise Theoretical Report

Prepared according to the Module 4 Robot Framework topics in the prescribed course sheet.

4.1 Introduction to Robot Framework

Robot Framework is an open-source automation framework designed around a readable, keyword-driven approach. It is widely used for acceptance testing, acceptance test-driven development and robotic process automation. Its architecture is extensible: libraries provide implementation for browser, API, database and other operations while test cases describe the intended behavior.

Key Theoretical Concepts

- **Keyword-driven approach:** Test cases are built from keywords. Built-in keywords provide general operations and external libraries add domain-specific capabilities.
- **Readable syntax:** Robot Framework uses a tabular, human-readable syntax that can be understood by both technical and non-technical stakeholders.
- **Test data:** Variables, templates and resource files help separate data from test logic.
- **Extensibility:** Python and other supported technologies can be used to implement custom libraries and keywords.
- **Reporting:** Robot Framework produces structured XML output and human-readable HTML reports and logs.

Illustrative Example

```
*** Test Cases ***
Basic Robot Test
    Log    Hello Robot Framework
    Should Be Equal    ${1}    ${1}
```

4.2 Environment Setup and RIDE

Robot Framework can be installed through Python's package manager. RIDE (Robot Framework Integrated Development Environment) provides an editor-oriented environment for creating and maintaining Robot Framework tests. A modern workflow may also use VS Code, PyCharm or another editor, while the robot command remains the standard execution mechanism.

Key Theoretical Concepts

- **Installation:** The core package can be installed using pip. Additional libraries are installed separately according to the automation requirement.
- **RIDE:** RIDE provides graphical support for editing Robot Framework suites and keywords.
- **Libraries:** SeleniumLibrary is used for web UI automation; other libraries can support APIs, databases or custom systems.
- **Verification:** Running robot --version verifies the installation and version.
- **Project structure:** A maintainable project commonly separates tests, resources, libraries and result directories.

Illustrative Example

```
python -m pip install robotframework
robot --version

# Web automation:
python -m pip install robotframework-seleniumlibrary
```

4.3 Basic Syntax and Keywords

Robot Framework files are organized into sections such as Settings, Variables, Test Cases and Keywords. The framework uses keywords as the fundamental reusable units of automation.

Key Theoretical Concepts

- **Settings:** Used for documentation, libraries, resources, setup and teardown.
- **Variables:** Stores scalar, list and dictionary variables that can be reused by tests.

- **Test Cases:** Contains executable test scenarios.
- **Keywords:** Contains user-defined reusable actions.
- **Arguments:** User-defined keywords can accept arguments, making them reusable for different inputs.
- **Assertions:** Built-in keywords such as Should Be Equal and Should Contain perform validation.

Illustrative Example

```
*** Variables ***
${NAME}    Saheli

*** Test Cases ***
Verify Name
    Should Be Equal    ${NAME}    Saheli

*** Keywords ***
Log Greeting
    [Arguments]    ${person}
    Log    Hello ${person}
```

4.4 Keyword-Driven Test Cases

Keyword-driven automation separates the test intent from implementation details. Instead of writing low-level commands repeatedly, a test can call meaningful business-oriented keywords.

Key Theoretical Concepts

- **Reusable actions:** A keyword can represent an action such as Login User or Search Product.
- **Abstraction:** Low-level Selenium operations can remain hidden inside keywords.
- **Maintainability:** Changing an implementation keyword can update many test cases at once.
- **Arguments:** Parameterized keywords can handle different users, products or values.
- **Layering:** Project-level keywords can call library keywords and other user-defined keywords.

Illustrative Example

```
*** Keywords ***
Login User
    [Arguments]    ${username}    ${password}
    Log    Logging in as ${username}
    Log    Password supplied securely

*** Test Cases ***
Login Test
    Login User    testuser    secret
```

4.5 Data-Driven Test Cases

Data-driven testing executes the same test logic against multiple input sets. Robot Framework supports templates and structured test data so that one keyword implementation can be reused for many rows.

Key Theoretical Concepts

- **Template:** The [Template] setting identifies the keyword that receives each row of data.
- **Examples:** Each test row supplies a different input combination.
- **Coverage:** Data-driven execution improves input coverage without duplicating test logic.
- **Separation:** Test data can be maintained separately from implementation keywords.
- **Use cases:** Login combinations, search terms, form values and validation boundaries are common examples.

Illustrative Example

```
*** Test Cases ***
Validate Values
    [Template]    Check Value
    Robot
    Selenium
    Python

*** Keywords ***
Check Value
    [Arguments]    ${value}
    Should Not Be Empty    ${value}
```

4.6 Command Line and Execution Options

Robot Framework suites are normally executed from the command line using the robot command. Command-line options can select tests, tags, output locations and logging behavior, making the framework suitable for local execution and continuous integration.

Key Theoretical Concepts

- **Full suite:** robot tests/ runs the available suite files under the selected path.
- **Single test:** --test selects a test case by name.
- **Tags:** --include and --exclude allow selective execution.
- **Output directory:** -d stores output.xml, log.html and report.html in a chosen directory.
- **Variables:** -v supplies command-line variables.
- **Dry run:** --dryrun can validate parsing and keyword matching without performing normal test actions.

Illustrative Example

```
# Examples
robot tests/
robot --test "Login Test" tests/
robot --include smoke -d results tests/
robot -v BROWSER:chrome tests/
```

4.7 Selenium Library and Browser Automation

SeleniumLibrary is a Robot Framework test library built on Selenium WebDriver. It provides keywords for opening browsers, navigating pages, locating elements, entering text, clicking controls and validating page content.

Key Theoretical Concepts

- **Locators:** Elements can be identified with IDs, names, CSS selectors, XPath and other locator strategies.
- **Browser control:** Keywords such as Open Browser, Close All Browsers and Go To Url manage browser sessions.
- **Element interaction:** Input Text, Click Element, Select From List and related keywords perform UI actions.
- **Synchronization:** Wait Until Element Is Visible and similar keywords help handle dynamic pages.
- **Validation:** Title Should Be, Page Should Contain and element-state assertions validate UI behavior.

Illustrative Example

```
*** Settings ***
Library    SeleniumLibrary

*** Test Cases ***
Selenium Demo
    Open Browser    https://www.selenium.dev/    chrome
    Title Should Be    Selenium
    Wait Until Page Contains    Selenium
    [Teardown]    Close All Browsers
```

4.8 Execution Options and Browser/Platform Coverage

Robot Framework execution can be parameterized so that the same suite can run against different browsers, environments and configurations. Tags and command-line variables are commonly used for selecting execution subsets.

Key Theoretical Concepts

- **Browser parameter:** The browser name can be supplied through a Robot variable.
- **Environment parameter:** URLs and environment-specific values should be externalized rather than hard-coded.
- **Tags:** Smoke, regression and feature tags help create focused test runs.
- **Parallel execution:** Parallel execution can be achieved through compatible execution tooling when required.
- **Artifacts:** Output XML, HTML report and log files provide evidence of execution.

Illustrative Example

```
*** Variables ***
${BROWSER}    chrome
${URL}        https://www.selenium.dev/

*** Test Cases ***
Parameterized Browser Test
    Open Browser    ${URL}    ${BROWSER}
    Title Should Be Selenium
    [Teardown]    Close All Browsers

# Run:
# robot -v BROWSER:firefox suite.robot
```

4.9 Sauce Labs Integration

Sauce Labs is a cloud-based testing platform that can provide remote browser and device environments. Robot Framework can connect through Selenium Remote WebDriver using the credentials and capabilities required by the service.

Key Theoretical Concepts

- **Remote WebDriver:** Instead of launching a local browser, the test creates a session on a remote provider.
- **Capabilities:** Browser, operating system and other execution details are supplied through capabilities/options.
- **Credentials:** Usernames, access keys and tokens should be stored as environment variables or secure CI secrets.
- **Scalability:** Cloud execution can provide access to multiple browser and platform combinations.
- **CI integration:** Remote tests can be triggered as part of Jenkins or other CI pipelines.

Illustrative Example

```
*** Settings ***
Library    SeleniumLibrary

*** Variables ***
${REMOTE_URL}    ${SAUCE_REMOTE_URL}

*** Test Cases ***
Remote Execution Pattern
    Log    Remote Selenium endpoint: ${REMOTE_URL}
    # Provider-specific browser capabilities are configured
    # according to the current Sauce Labs setup.
```

4.10 Jenkins Scheduling and CI Execution

Jenkins is a continuous integration server that can execute Robot Framework tests automatically. A Jenkins job can install dependencies, execute tagged suites, archive Robot Framework artifacts and schedule builds.

Key Theoretical Concepts

- **Build step:** The Jenkins agent runs the robot command in the workspace.
- **Scheduled trigger:** Cron-style or other Jenkins triggers can start tests at scheduled times.
- **Artifacts:** output.xml, log.html and report.html can be archived for later inspection.
- **Parameters:** Browser, environment URL and tags can be supplied as build parameters.
- **Failure handling:** The CI job can mark the build as failed when Robot Framework returns a non-zero exit code.

Illustrative Example

```
# Jenkins build command

python -m pip install -r requirements.txt
robot --include smoke -d results tests/

# Archive:
# results/output.xml
# results/log.html
# results/report.html
```

4.11 User-Defined Keywords and Resource Files

User-defined keywords allow teams to create domain-specific actions. Resource files allow these keywords, variables and settings to be shared by multiple suites.

Key Theoretical Concepts

- **Keyword abstraction:** A keyword can represent a business action rather than a technical action.
- **Resource import:** Resource files are imported using the Resource setting.
- **Centralization:** Common login, navigation and validation logic can be maintained in one location.
- **Arguments:** Keywords can accept parameters for reuse.
- **Organization:** Separate resource files can be created for pages, common actions and test data.

Illustrative Example

```
# resources/common.resource

*** Keywords ***
Open Application
    [Arguments]    ${url}
    Log    Opening ${url}

Verify Text
    [Arguments]    ${actual}    ${expected}
    Should Be Equal    ${actual}    ${expected}

# suite.robot
*** Settings ***
Resource    resources/common.resource
```

4.12 Setup and Teardown

Setup and teardown define actions that run before and after suites, tests or keywords. They are important for consistent initialization and cleanup.

Key Theoretical Concepts

- **Suite Setup:** Runs once before the suite starts.
- **Suite Teardown:** Runs after the suite finishes.
- **Test Setup:** Runs before each test.

- **Test Teardown:** Runs after each test, including when the test fails.
- **Cleanup:** Browser closing, temporary-file deletion and state reset are common teardown tasks.

Illustrative Example

```
*** Settings ***
Suite Setup      Log    Suite started
Suite Teardown   Log    Suite finished
Test Setup       Log    Test started
Test Teardown    Log    Test finished

*** Test Cases ***
Lifecycle Demo
    Log    Test body
```

4.13 Page Object Model

Page Object Model (POM) is a design pattern in which page-specific locators and actions are grouped into page-oriented abstractions. In Robot Framework, the same idea can be implemented through resource files, page-level keywords or custom libraries.

Key Theoretical Concepts

- **Encapsulation:** Locators and page behavior are kept together.
- **Reusability:** Multiple tests can use the same page keywords.
- **Maintainability:** UI changes can often be handled in the page abstraction instead of every test.
- **Readable tests:** Tests can call business-level actions such as Login or Open Cart.
- **Separation:** Test intent is separated from browser implementation details.

Illustrative Example

```
*** Settings ***
Library    SeleniumLibrary

*** Keywords ***
Open Login Page
    Open Browser    https://example.com/login    chrome

Enter Username
    [Arguments]    ${username}
    Input Text    id=username    ${username}

Enter Password
    [Arguments]    ${password}
    Input Text    id=password    ${password}
```

4.14 Gherkin-Style Readability

Robot Framework can be written in a Gherkin-like Given/When/Then style by creating user-defined keywords with readable names. This improves communication between technical and business stakeholders while retaining Robot Framework's keyword model.

Key Theoretical Concepts

- **Given:** Represents the initial condition.
- **When:** Represents the action.
- **Then:** Represents the expected result.
- **And:** Can be represented through additional readable keywords.
- **Benefit:** The resulting test communicates behavior without exposing every low-level automation detail.

Illustrative Example

```
*** Keywords ***
Given User Opens Selenium Website
    Open Browser    https://www.selenium.dev/    chrome

When User Opens Documentation
    Click Link    Documentation

Then Documentation Page Is Displayed
    Wait Until Page Contains    Selenium Documentation

*** Test Cases ***
Documentation Navigation
    Given User Opens Selenium Website
    When User Opens Documentation
    Then Documentation Page Is Displayed
    [Teardown]    Close All Browsers
```

Overall Module 4 Theoretical Summary

Robot Framework provides a readable, keyword-driven foundation for test automation. The module progresses from installation and syntax to keyword-driven and data-driven testing, Selenium browser automation, execution control, cloud execution, CI scheduling and maintainable design using resource files, setup/teardown and Page Object concepts. Gherkin-style naming can further improve readability and communication.

Topic	Covered
Introduction / Environment / RIDE	4.1–4.2
Syntax / Keywords / Data Driven	4.3–4.5
Command Line / Selenium / Execution	4.6–4.8
Sauce Labs / Jenkins	4.9–4.10
Resources / Setup & Teardown / POM / Gherkin	4.11–4.14